Hyrje në gjuhën C

Alion Emini

kurseinformatike.com

Përmbajtja

1. Pse po mësojmë C-në?
2. Programe në C
3. Funksioni `printf`
4. Funksioni: `scanf`
5. Operatorët
6. Ushtrime/Detyra
7. Përmbledhje

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

Nga algoritmi te C-ja

- Kapitulli 1: si mendojmë algoritmikisht.

Nga algoritmi te C-ja

- Kapitulli 1: si mendojmë algoritmikisht.
- Tani: si i kthejmë këto ide në programe reale.

Nga algoritmi te C-ja

- Kapitulli 1: si mendojmë algoritmikisht.
- Tani: si i kthejmë këto ide në programe reale.
- Zgjedhja jonë: **gjuha C** një nga gjuhët më me ndikim.

Çfarë është gjuha C?

Gjuha C

C është gjuhë programimi:

- relativisht e ulët (krahasuar me Python, Java);
- *proceduriale*;
- projektuar fillimisht për sisteme operative dhe software pranë harduerit;
- me qasje efikase në memorie dhe portabilitet të lartë.

Pse nuk mjafton vetëm Python-i?

- C ju jep pamje **"poshtë kapakut"**:

Pse nuk mjafton vetëm Python-i?

- C ju jep pamje **"poshtë kapakut"**:
  - llojet e të dhënave;

Pse nuk mjafton vetëm Python-i?

- C ju jep pamje **"poshtë kapakut"**:
  - llojet e të dhënave;
  - adresat në memorie;

Pse nuk mjafton vetëm Python-i?

- C ju jep pamje **"poshtë kapakut"**:
  - llojet e të dhënave;
  - adresat në memorie;
  - pointer-at dhe menaxhimin manual të burimeve.

Pse nuk mjafton vetëm Python-i?

- C ju jep pamje **"poshtë kapakut"**:
  - llojet e të dhënave;
  - adresat në memorie;
  - pointer-at dhe menaxhimin manual të burimeve.
- Shumë gjuhë moderne dhe sisteme janë ndërtuar me/mbi C.

Pse nuk mjafton vetëm Python-i?

- C ju jep pamje **"poshtë kapakut"**:
  - llojet e të dhënave;
  - adresat në memorie;
  - pointer-at dhe menaxhimin manual të burimeve.
- Shumë gjuhë moderne dhe sisteme janë ndërtuar me/mbi C.
- Kush kupton C-në, kupton më mirë edhe gjuhët me nivel më të lartë.

Mesazh për studentin

Nëse e mësoni C-në me durim

- kuptoni kod real sistemi (UNIX/Linux, biblioteka, driver-a);

Mesazh për studentin

Nëse e mësoni C-në me durim

- kuptoni kod real sistemi (UNIX/Linux, biblioteka, driver-a);
- kaloni më lehtë në C++, Java, C#, Go, Rust, etj.;

Mesazh për studentin

Nëse e mësoni C-në me durim

- kuptoni kod real sistemi (UNIX/Linux, biblioteka, driver-a);
- kaloni më lehtë në C++, Java, C#, Go, Rust, etj.;
- fitoni disiplinë: jeni më të kujdesshëm me memorie dhe gabime.

Nga Algol te BCPL dhe B

- **Algol 60**: struktura kontrolli, ndikim i madh akademik.

Nga Algol te BCPL dhe B

- **Algol 60**: struktura kontrolli, ndikim i madh akademik.
- **BCPL**: gjuhë e thjeshtë për sisteme.

Nga Algol te BCPL dhe B

- **Algol 60**: struktura kontrolli, ndikim i madh akademik.
- **BCPL**: gjuhë e thjeshtë për sisteme.
- **B**: zhvilluar në Bell Labs, përdorur për UNIX-in e hershëm.

Nga Algol te BCPL dhe B

- **Algol 60**: struktura kontrolli, ndikim i madh akademik.
- **BCPL**: gjuhë e thjeshtë për sisteme.
- **B**: zhvilluar në Bell Labs, përdorur për UNIX-in e hershëm.
- Problemi:

Nga Algol te BCPL dhe B

- **Algol 60**: struktura kontrolli, ndikim i madh akademik.
- **BCPL**: gjuhë e thjeshtë për sisteme.
- **B**: zhvilluar në Bell Labs, përdorur për UNIX-in e hershëm.
- Problemi:
  - Assembly: shumë i varur nga procesori.

Nga Algol te BCPL dhe B

- **Algol 60**: struktura kontrolli, ndikim i madh akademik.
- **BCPL**: gjuhë e thjeshtë për sisteme.
- **B**: zhvilluar në Bell Labs, përdorur për UNIX-in e hershëm.
- Problemi:
  - Assembly: shumë i varur nga procesori.
  - BCPL/B: shumë të kufizuara për harduerë të rinj.

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.
- Zgjeron B për PDP-11 $\Rightarrow$ NB (*New B*) $\Rightarrow$ **C**.

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.
- Zgjeron B për PDP-11 $\Rightarrow$ NB (*New B*) $\Rightarrow$ **C**.
- Rreth 1973: pjesa më e madhe e **UNIX** rishkruhet në C.

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.
- Zgjeron B për PDP-11 $\Rightarrow$ NB (*New B*) $\Rightarrow$ **C**.
- Rreth 1973: pjesa më e madhe e **UNIX** rishkruhet në C.
- Pasoja:

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.
- Zgjeron B për PDP-11 $\Rightarrow$ NB (*New B*) $\Rightarrow$ **C**.
- Rreth 1973: pjesa më e madhe e **UNIX** rishkruhet në C.
- Pasoja:
  - OS nuk lidhet vetëm me Assembly;

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.
- Zgjeron B për PDP-11 $\Rightarrow$ NB (*New B*) $\Rightarrow$ **C**.
- Rreth 1973: pjesa më e madhe e **UNIX** rishkruhet në C.
- Pasoja:
  - OS nuk lidhet vetëm me Assembly;
  - me një kompilues C, UNIX portohet në makina të tjera;

Lindja e C-së në Bell Labs

- Fillimi i viteve 1970, **Dennis Ritchie** në Bell Labs.
- Zgjeron B për PDP-11 $\Rightarrow$ NB (*New B*) $\Rightarrow$ **C**.
- Rreth 1973: pjesa më e madhe e **UNIX** rishkruhet në C.
- Pasoja:
  - OS nuk lidhet vetëm me Assembly;
  - me një kompilues C, UNIX portohet në makina të tjera;
  - C bëhet mjet kyç për zhvillim sistemesh.

Pse ka rëndësi kjo histori?

- C është krijuar për probleme reale sistemi, jo si lodër teorike.

Pse ka rëndësi kjo histori?

- C është krijuar për probleme reale sistemi, jo si lodër teorike.
- Filozofia e saj: efikasitet, kontroll, thjeshtësi në bërthamë.

Pse ka rëndësi kjo histori?

- C është krijuar për probleme reale sistemi, jo si lodër teorike.
- Filozofia e saj: efikasitet, kontroll, thjeshtësi në bërthamë.
- Shumë sisteme moderne (Linux, pjesë të Windows-it, driver-a) janë ende në C.

Nga K&R te C89/C90

- 1978: libri *The C Programming Language* (Kernighan & Ritchie).

Nga K&R te C89/C90

- 1978: libri *The C Programming Language* (Kernighan & Ritchie).
- Shërben si "standard jozyrtar" (*K&R C*).

Nga K&R te C89/C90

- 1978: libri *The C Programming Language* (Kernighan & Ritchie).
- Shërben si "standard jozyrtar" (*K&R C*).
- Problem:

Nga K&R te C89/C90

- 1978: libri *The C Programming Language* (Kernighan & Ritchie).
- Shërben si "standard jozyrtar" (*K&R C*).
- Problem:
  - disa veçori të paqarta;

Nga K&R te C89/C90

- 1978: libri *The C Programming Language* (Kernighan & Ritchie).
- Shërben si "standard jozyrtar" (*K&R C*).
- Problem:
  - disa veçori të paqarta;
  - kompilues të ndryshëm, dialekte të ndryshme;

Nga K&R te C89/C90

- 1978: libri *The C Programming Language* (Kernighan & Ritchie).
- Shërben si "standard jozyrtar" (*K&R C*).
- Problem:
  - disa veçori të paqarta;
  - kompilues të ndryshëm, dialekte të ndryshme;
  - humbje e portabilitetit.

C89 / C90

Standardizimi

- 1989: **ANSI C** (C89).

Program në C të pastër ⇒ respektim i standardit, jo dialekt lokal.

C89 / C90

Standardizimi

- 1989: **ANSI C** (C89).
- 1990: miratohet si **ISO/IEC 9899:1990** (C90).

Program në C të pastër ⇒ respektim i standardit, jo dialekt lokal.

C89 / C90

Standardizimi

- 1989: **ANSI C** (C89).
- 1990: miratohet si **ISO/IEC 9899:1990** (C90).
- Përkufizojnë qartë:

Program në C të pastër ⇒ respektim i standardit, jo dialekt lokal.

C89 / C90

Standardizimi

- 1989: **ANSI C** (C89).
- 1990: miratohet si **ISO/IEC 9899:1990** (C90).
- Përkufizojnë qartë:
  - sintaksën dhe semantikën;

Program në C të pastër ⇒ respektim i standardit, jo dialekt lokal.

C89 / C90

Standardizimi

- 1989: **ANSI C** (C89).
- 1990: miratohet si **ISO/IEC 9899:1990** (C90).
- Përkufizojnë qartë:
  - sintaksën dhe semantikën;
  - bibliotekën standarde;

Program në C të pastër ⇒ respektim i standardit, jo dialekt lokal.

C89 / C90

Standardizimi

- 1989: **ANSI C** (C89).
- 1990: miratohet si **ISO/IEC 9899:1990** (C90).
- Përkufizojnë qartë:
  - sintaksën dhe semantikën;
  - bibliotekën standarde;
  - bazën e përbashkët për kompiluesit.

Program në C të pastër ⇒ respektim i standardit, jo dialekt lokal.

Versionet pasuese

- **C99**: `long long`, komente `//`, përmirësime në `for`, inicializues, etj.

Versionet pasuese

- **C99**: `long long`, komente `//`, përmirësime në `for`, inicializues, etj.
- **C11**: mbështetje për multithreading, tipa atomikë, etj.

Versionet pasuese

- **C99**: `long long`, komente `//`, përmirësime në `for`, inicializues, etj.
- **C11**: mbështetje për multithreading, tipa atomikë, etj.
- **C17/C18**: korrigjime, pa ndryshime të mëdha.

Versionet pasuese

- **C99**: `long long`, komente `//`, përmirësime në `for`, inicializues, etj.
- **C11**: mbështetje për multithreading, tipa atomikë, etj.
- **C17/C18**: korrigjime, pa ndryshime të mëdha.
- **C23**: rafinime të reja (kompiluesit modernë e ndjekin gradualisht).

Çfarë na duhet në këtë kurs?

Shënim

- Përdorim një **nën-bërthamë** C-je të vlefshme në C89, C99, C11, C17, C23.

Çfarë na duhet në këtë kurs?

Shënim

- Përdorim një **nën-bërthamë** C-je të vlefshme në C89, C99, C11, C17, C23.
- Theksojmë vetëm ndryshimet që ju prekin realisht.

Çfarë na duhet në këtë kurs?

Shënim

- Përdorim një **nën-bërthamë** C-je të vlefshme në C89, C99, C11, C17, C23.
- Theksojmë vetëm ndryshimet që ju prekin realisht.
- Shkruajmë kod që kompilohet në çdo kompilues modern C.

Gjuhet që duken "si C"

- **C++**: C me klasa, OOP, template, abstraksione të larta.

Gjuhet që duken "si C"

- **C++**: C me klasa, OOP, template, abstraksione të larta.
- **Java**: sintaksë C-like, JVM, garbage collection.

Gjuhet që duken "si C"

- **C++**: C me klasa, OOP, template, abstraksione të larta.
- **Java**: sintaksë C-like, JVM, garbage collection.
- **C#**: e frymëzuar nga C++/Java, përdorur gjerësisht në .NET.

Gjuhet që duken "si C"

- **C++**: C me klasa, OOP, template, abstraksione të larta.
- **Java**: sintaksë C-like, JVM, garbage collection.
- **C#**: e frymëzuar nga C++/Java, përdorur gjerësisht në .NET.
- **JavaScript**: sintaksë familjare (if, for, { }), edhe pse tjetër filozofi.

Gjuhet që duken "si C"

- **C++**: C me klasa, OOP, template, abstraksione të larta.
- **Java**: sintaksë C-like, JVM, garbage collection.
- **C#**: e frymëzuar nga C++/Java, përdorur gjerësisht në .NET.
- **JavaScript**: sintaksë familjare (if, for, { }), edhe pse tjetër filozofi.
- **Perl**, **PHP**, **Go**, **Rust**, etj.: elemente sintaksore nga C.

Mesazhi kryesor

Nëse di C...

- kalimi në shumë gjuhë të tjera bëhet më i lehtë;

Mesazhi kryesor

Nëse di C...

- kalimi në shumë gjuhë të tjera bëhet më i lehtë;
- shumica e strukturave të kontrollit duken të njohura;

Mesazhi kryesor

Nëse di C...

- kalimi në shumë gjuhë të tjera bëhet më i lehtë;
- shumica e strukturave të kontrollit duken të njohura;
- kupton burimin e shumë idiomave moderne në programim.

Arsyet kryesore

1. **Kupton si funksionon kompjuteri**: lloje, memorie, pointer-a, nivel i ulët.

Arsyet kryesore

1. **Kupton si funksionon kompjuteri**: lloje, memorie, pointer-a, nivel i ulët.
2. **Gjuhë kyçe për sistemet**: OS, driver-a, kompajlerë, biblioteka sistemike.

Arsyet kryesore

1. **Kupton si funksionon kompjuteri**: lloje, memorie, pointer-a, nivel i ulët.
2. **Gjuhë kyçe për sistemet**: OS, driver-a, kompajlerë, biblioteka sistemike.
3. **Portabilitet dhe jetëgjatësi**: kod që vrapon kudo, bazë ekzistuese gjigante.

Arsyet kryesore

1. **Kupton si funksionon kompjuteri**: lloje, memorie, pointer-a, nivel i ulët.
2. **Gjuhë kyçe për sistemet**: OS, driver-a, kompajlerë, biblioteka sistemike.
3. **Portabilitet dhe jetëgjatësi**: kod që vrapon kudo, bazë ekzistuese gjigante.
4. **Bazë e fortë për gjuhë të tjera**.

Arsyet kryesore

1. **Kupton si funksionon kompjuteri**: lloje, memorie, pointer-a, nivel i ulët.
2. **Gjuhë kyçe për sistemet**: OS, driver-a, kompajlerë, biblioteka sistemike.
3. **Portabilitet dhe jetëgjatësi**: kod që vrapon kudo, bazë ekzistuese gjigante.
4. **Bazë e fortë për gjuhë të tjera**.
5. **Thjeshtësi në bërthamë**: pak rregulla, shumë fuqi.

Paralajmërim i ndershëm

C nuk fal shumë...

- gabimet me pointer-a mund të prishin gjithë programin;

Pikërisht për këtë, C ju rrit si programues më të kujdesshëm.

Paralajmërim i ndershëm

C nuk fal shumë...

- gabimet me pointer-a mund të prishin gjithë programin;
- nuk ka garbage collector;

Pikërisht për këtë, C ju rrit si programues më të kujdesshëm.

Paralajmërim i ndershëm

C nuk fal shumë...

- gabimet me pointer-a mund të prishin gjithë programin;
- nuk ka garbage collector;
- kompiluesi shpesh lejon "gjëra të rrezikshme".

Pikërisht për këtë, C ju rrit si programues më të kujdesshëm.

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

.c ⇒ program ekzekutues

Procesi kryesor:

- **Preprocesimi**

Zakonisht, gcc i bën këto automatikisht njëri pas tjetrit.

.c ⇒ program ekzekutues

Procesi kryesor:

- **Preprocesimi**
- **Kompilimi**

Zakonisht, gcc i bën këto automatikisht njëri pas tjetrit.

.c ⇒ program ekzekutues

Procesi kryesor:

- **Preprocesimi**
- **Kompilimi**
- **Asamblimi**

Zakonisht, gcc i bën këto automatikisht njëri pas tjetrit.

.c ⇒ program ekzekutues

Procesi kryesor:

- **Preprocesimi**
- **Kompilimi**
- **Asamblimi**
- **Linkimi**

Zakonisht, gcc i bën këto automatikisht njëri pas tjetrit.

Preprocesimi

- Trajton rreshta që fillojnë me #.

Preprocesimi

- Trajton rreshta që fillojnë me #.
- Shembuj:

Preprocesimi

- Trajton rreshta që fillojnë me #.
- Shembuj:
  - `#include <stdio.h>`

Preprocesimi

- Trajton rreshta që fillojnë me #.
- Shembuj:
  - `#include <stdio.h>`
  - `#define PI 3.14`

Preprocesimi

- Trajton rreshta që fillojnë me #.
- Shembuj:
  - `#include <stdio.h>`
  - `#define PI 3.14`
- Fut header-at, zëvendëson makrot.

Preprocesimi

- Trajton rreshta që fillojnë me #.
- Shembuj:
  - `#include <stdio.h>`
  - `#define PI 3.14`
- Fut header-at, zëvendëson makrot.
- Rezultati: kod burimor i zgjeruar.

Kompilimi dhe asamblimi

- Kontroll sintakse dhe tipash.

Kompilimi dhe asamblimi

- Kontroll sintakse dhe tipash.
- Përkthim në assembly / kod makine.

Kompilimi dhe asamblimi

- Kontroll sintakse dhe tipash.
- Përkthim në assembly / kod makine.
- Gjenerohet **file objekt** `.o` (kod makine, jo ende program i plotë).

Kompilimi dhe asamblimi

- Kontroll sintakse dhe tipash.
- Përkthim në assembly / kod makine.
- Gjenerohet **file objekt** `.o` (kod makine, jo ende program i plotë).
- Asamblimi shpesh është i integruar në të njëjtën komandë.

Linkimi

- Bashkon:

Linkimi

- Bashkon:
  - file `.o`;

Linkimi

- Bashkon:
  - file `.o`;
  - funksione nga bibliotekat (p.sh. `printf` nga `stdio`).

Linkimi

- Bashkon:
  - file `.o`;
  - funksione nga bibliotekat (p.sh. `printf` nga `stdio`).
- Zgjidh simbolet dhe krijon **ekzekutuesin final** (`program`, `program.exe`).

Shembull: një file

```
gcc -c program.c -o program.o     /* kompilim -> .o */
gcc program.o -o program          /* linkim -> ekzekutues */
./program
```

Shembull: disa file

```
gcc -c main.c -o main.o
gcc -c functions.c -o functions.o
gcc main.o functions.o -o myprogram
./myprogram
```

Skeleti bazë

```
#include <stdio.h>

int main(void) {
    /* Këtu shkruajmë komandat */
    return 0;
}
```

Çfarë po ndodh këtu?

- `#include <stdio.h>`: direktivë paraprosesori, deklaron `printf`, `scanf`, etj.

Çfarë po ndodh këtu?

- `#include <stdio.h>`: direktivë paraprosesori, deklaron `printf`, `scanf`, etj.
- `int main(void)`: funksioni kryesor, aty fillon programi.

Çfarë po ndodh këtu?

- `#include <stdio.h>`: direktivë paraprosesori, deklaron `printf`, `scanf`, etj.
- `int main(void)`: funksioni kryesor, aty fillon programi.
- `{ }`: trupi i funksionit (blloku i deklaratave).

Çfarë po ndodh këtu?

- `#include <stdio.h>`: direktivë paraprosesori, deklaron `printf`, `scanf`, etj.
- `int main(void)`: funksioni kryesor, aty fillon programi.
- `{ }`: trupi i funksionit (blloku i deklaratave).
- `return 0;`: njofton sistemin operativ se programi përfundoi me sukses.

Tre shtyllat e programit në C

Shënim

1. **Direktivat** (`#include`, `#define`, ...)

Pjesa tjetër e kursit: t'i përdorim mirë këto tre.

Tre shtyllat e programit në C

Shënim

1. **Direktivat** (`#include`, `#define`, ...)
2. **Funksionet** (`main`, funksionet tona)

Pjesa tjetër e kursit: t'i përdorim mirë këto tre.

Tre shtyllat e programit në C

Shënim

1. **Direktivat** (`#include`, `#define`, ...)
2. **Funksionet** (`main`, funksionet tona)
3. **Deklaratat** (komandat brenda funksioneve)

Pjesa tjetër e kursit: t'i përdorim mirë këto tre.

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

printf: forma bazë

```
printf("format string", vlera1, vlera2, ...);
```

- Argumenti i parë: **string formati**.

printf: forma bazë

```
printf("format string", vlera1, vlera2, ...);
```

- Argumenti i parë: **string formati**.
- Mund të përmbajë:

printf: forma bazë

```
printf("format string", vlera1, vlera2, ...);
```

- Argumenti i parë: **string formati**.
- Mund të përmbajë:
  - tekst normal;

printf: forma bazë

```
printf("format string", vlera1, vlera2, ...);
```

- Argumenti i parë: **string formati**.
- Mund të përmbajë:
  - tekst normal;
  - specifikues formati që fillojnë me %.

Shembull

```
int a = 10;
float x = 3.5f;

printf("a = %d, x = %f\n", a, x);
```

Output:

```
a = 10, x = 3.500000
```

Specifikuesit kryesorë

%d	numër i plotë (`int`)
%f	numër real (`float`/`double`)
%e	formë eksponenciale
%g	zgjedh %f ose %e automatikisht
%c	një karakter
%s	string (varg karakteresh)

Gjerësi fushe dhe precizion

```
int i = 40;
float x = 839.21f;

printf("%d|%5d|%-5d|%5.3d|\n", i, i, i, i);
printf("%10.3f|%10.3e|%-10g|\n", x, x, x);
```

Leximi

- %5d : të paktën 5 karaktere.
- %-5d: justifikuar majtas.
- %5.3d: 3 shifra, fusha 5.
- %10.3f: fusha 10, 3 pas presjes.

Gabimet klasike me printf

```
/* Mospërputhje numrash */
printf("%d %d\n", a);        /* GABIM */

/* Mospërputhje tipash */
printf("%f\n", a);           /* GABIM: %f pret float/double */
```

Shënim

Kompiluesi mund të mos ankohet, por output del i gabuar.

Karaktere speciale në string

- Përdorim \ + simbol.
- Më të zakonshmet:

`\n`	rresht i ri
`\t`	tab horizontale
`\\`	një backslash
`\"`	thonjëza të dyfishta
`\'`	thonjëza të vetme
`\a`	sinjal tingulli (në disa sisteme)

Shembull

```
printf("Item\tPrice\n");
printf("\"Hello!\"\n");
printf("Backslash: \\\n");
```

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

scanf: forma bazë

```
int i;
scanf("%d", &i);
```

- Përdor *format string* si `printf`.

scanf: forma bazë

```
int i;
scanf("%d", &i);
```

- Përdor *format string* si `printf`.
- Lexon nga hyrja standarde (zakonisht tastiera).

Shembull me disa vlera

```
int i, j;
float x, y;

scanf("%d%d%f%f", &i, &j, &x, &y);
```

Pse duhet &?

- scanf duhet të dijë **ku** në memorie të vendosë vlerën.
- Për këtë i japim **adresën** me &.

```
scanf("\%d", \&num); \* sakte *\
scanf("\%d", num); \* gabim *\
```

Specifikuesit kryesorë për scanf

%d	lexon int
%f	lexon float
%lf	lexon double
%c	një karakter
%s	string (deri në hapësirë)

Kujdes me scanf

Këshilla

- %s ndalon te hapësira ⇒ jo për fjali të plota.

Kujdes me scanf

Këshilla

- %s ndalon te hapësira ⇒ jo për fjali të plota.
- Kontrolloni vlerën e kthyer nga scanf (sa elementë lexoi).

Kujdes me scanf

Këshilla

- %s ndalon te hapësira ⇒ jo për fjali të plota.
- Kontrolloni vlerën e kthyer nga scanf (sa elementë lexoi).
- Harrimi i & ose formati i gabuar ⇒ sjellje e rrezikshme.

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

Operatorët bazë

+	mbledhje	a + b
-	zbritje	a - b
*	shumëzim	a * b
/	pjesëtim	a / b
%	mbetje	a % b

Shembull me int

```
int a = 10, b = 3;

printf("%d\n", a / b);   /* 3  */
printf("%d\n", a % b);   /* 1  */
```

Shënim

- Me dy int: / heq pjesën dhjetore.
- a / 0 dhe a % 0 janë të ndaluara.

Forma të shkurtra dhe ++/–

```
i = i + 1;    /* e njejte me: */  i += 1;    i++;
j = j - 1;    /* e njejte me: */  j -= 1;    j--;
```

- ++i, i++: rrisin me 1.

Forma të shkurtra dhe ++/–

```
i = i + 1;    /* e njejte me: */  i += 1;    i++;
j = j - 1;    /* e njejte me: */  j -= 1;    j--;
```

- `++i`, `i++`: rrisin me 1.
- `-i`, `i-`: ulin me 1.

Prefix vs Postfix

Prefix:

```
int i = 1;
printf("%d\n", ++i);    /* shtyp 2, pastaj i = 2 */
```

Postfix:

```
int j = 1;
printf("%d\n", j++);    /* shtyp 1, pastaj j = 2 */
```

Shembull i kombinuar

```
int i = 1, j = 2, k;
k = ++i + j++;
/* ++i -> 2, j++ perdor 2 pastaj j=3, k = 4 */
```

Shënim

Mos shkruani shprehje shumë të ngatërruara me ++/−. Lexueshmëria > "truket".

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

Ushtrimi 1: Formati i datës

Shkruani një program që:

- lexon një datë nga përdoruesi në formatin `mm/dd/yyyy`;
- e shfaq të njëjtën datë në formatin `yyyymmdd` duke përdorur formatim të përshtatshëm me `printf`.

Ushtrimi 1: Pseudokodi

```
Start
Lexo month, day, year nga input ne formatin mm/dd/yyyy
Shfaq year, month, day ne formatin yyyymmdd
End
```

Ushtrimi 1: Kodi në C

```
#include <stdio.h>

int main(void) {
    int month, day, year;

    printf("Enter a date (mm/dd/yyyy): ");
    scanf("%d/%d/%d", &month, &day, &year);

    printf("You entered the date %04d%02d%02d\n", year, month,
        day);

    return 0;
}
```

Ushtrimi 2: Produkti

Programi duhet të:

- lexojë numrin e artikullit (`int`), çmimin për njësi (`double`) dhe datën e blerjes (`mm/dd/yyyy`);
- shfaqë të dhënat në kolona të rregulluara:
  - `Item`
  - `Unit Price` me 2 shifra pas presjes
  - `Purchase Date` në formatin `mm/dd/yyyy`

Ushtrimi 2: Pseudokodi

```
Start
Lexo item
Lexo price
Lexo month, day, year ne formatin mm/dd/yyyy
Shfaq "Item" dhe item
Shfaq "Unit Price" dhe price me 2 shifra pas presjes
Shfaq "Purchase Date" dhe daten ne formatin mm/dd/yyyy
End
```

Ushtrimi 2: Kodi në C

```
#include <stdio.h>
int main(void) {
    int item; double price;
    int month, day, year;

    printf("Enter item number: ");
    scanf("%d", &item);

    printf("Enter unit price: ");
    scanf("%lf", &price);

    printf("Enter purchase date (mm/dd/yyyy): ");
    scanf("%d/%d/%d", &month, &day, &year);

    printf("\nItem %d\n", item);
    printf("Unit Price $ %6.2f\n", price);
    printf("Purchase Date %02d/%02d/%04d\n", month, day, year);

    return 0;
}
```

Ushtrimi 3: Ndarja e ISBN-13

Shkruani një program që:

- lexon një ISBN-13 në formatin `xxx-x-xxx-xxxxx-x`;
- shfaq veçmas:
  - `GS1 prefix`
  - `Group identifier`
  - `Publisher code`
  - `Item number`
  - `Check digit`

Ushtrimi 3: Ndarja e ISBN-13 Pseudokodi

```
Start
Lexo gs1, group, publisher, item, check
    nga input ne formatin xxx-x-xxx-xxxxx-x
Shfaq "GS1 prefix" dhe gs1
Shfaq "Group identifier" dhe group
Shfaq "Publisher code" dhe publisher
Shfaq "Item number" dhe item
Shfaq "Check digit" dhe check
End
```

Ushtrimi 3: Kodi në C

```
#include <stdio.h>

int main(void) {
    int gs1, group, publisher, item, check;

    printf("Enter ISBN: ");
    scanf("%d-%d-%d-%d-%d", &gs1, &group, &publisher, &item, &check);

    printf("GS1 prefix: %d\n", gs1);
    printf("Group identifier: %d\n", group);
    printf("Publisher code: %d\n", publisher);
    printf("Item number: %d\n", item);
    printf("Check digit: %d\n", check);

    return 0;
}
```

Ushtrimi 4: Minutat nga mesnata

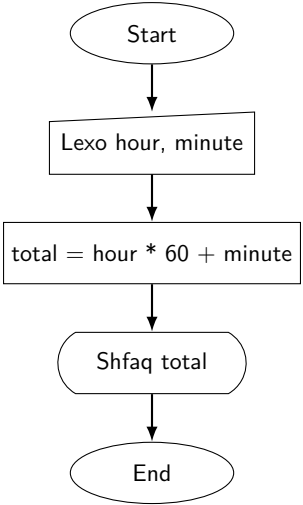
Shkruani një program që:

- lexon një kohë në formatin `hh:mm` (ora dhe minuta);
- llogarit numrin e minutave që kanë kaluar nga mesnata;
- e shfaq rezultatin.

Ushtrimi 4: Pseudokodi

```
Start
Lexo hour, minute ne formatin hh:mm
total = hour * 60 + minute
Shfaq "Minutes since midnight:" dhe total
End
```

Ushtrimi 4: Flowchart



Ushtrimi 4: Kodi në C

```
#include <stdio.h>

int main(void) {
    int hour, minute;
    int total;

    printf("Enter a 24-hour time (hh:mm): ");
    scanf("%d:%d", &hour, &minute);

    total = hour * 60 + minute;

    printf("Minutes since midnight: %d\n", total);

    return 0;
}
```

Ushtrimi 5: ++ dhe –

Shkruani një program që:

- lexon një numër të plotë `n`;
- shfaq vlerën fillestare të `n`;
- shfaq rezultatin e `n++` dhe vlerën pas tij;
- shfaq rezultatin e `++n` dhe vlerën përfundimtare të `n`.

Ushtrimi 5: ++ dhe – Pseudokodi

```
Start
Lexo n
Shfaq vleren fillestare te n
Shfaq vleren e n++ (perdor n, pastaj rrit)
Shfaq vleren aktuale te n
Shfaq vleren e ++n (rrit, pastaj perdor)
Shfaq vleren finale te n
End
```

Ushtrimi 5: Kodi në C

```c
#include <stdio.h>

int main(void) {
    int n;

    printf("Enter an integer: ");
    scanf("%d", &n);

    printf("Initial n: %d\n", n);
    printf("Using n++ (prints old n): %d\n", n++);
    printf("After n++: %d\n", n);
    printf("Using ++n (increments then prints): %d\n", ++n);
    printf("Final n: %d\n", n);

    return 0;
}
```

Ushtrimi 6: Operacione aritmetikore

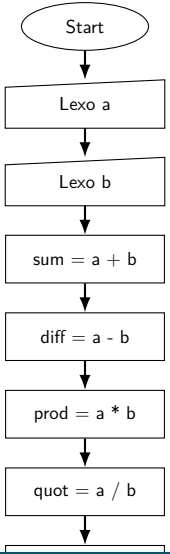
Shkruani një program që:

- lexon dy numra të plotë a dhe b;
- llogarit dhe shfaq:
  - shumën a + b;
  - diferencën a - b;
  - prodhimin a * b;
  - pjesëtimin e plotë a / b (b != 0);
  - mbetjen a % b.

Ushtrimi 6: Operacione aritmetikore Pseudokodi

```
Start
Lexo a
Lexo b
sum  = a + b
diff = a - b
prod = a * b
quot = a / b        (supozome b != 0)
rem  = a % b
Shfaq sum, diff, prod, quot, rem
End
```

Ushtrimi 6: Flowchart



Ushtrimi 6: Kodi në C

```
#include <stdio.h>
int main(void) {
    int a, b, sum, diff, prod, quot, rem;
    printf("Enter two integers (a b): ");
    scanf("%d%d", &a, &b);

    sum  = a + b;
    diff = a - b;
    prod = a * b;
    quot = a / b;   /* supozojme b != 0 */
    rem  = a % b;

    printf("Sum: %d\n", sum);
    printf("Difference: %d\n", diff);
    printf("Product: %d\n", prod);
    printf("Quotient: %d\n", quot);
    printf("Remainder: %d\n", rem);
    return 0;
}
```

Detyra 1: Formati i kohës

Shkruani një program që lexon një kohë në formatin `hh:mm` (24-orësh) dhe e shfaq:

- si `hhmm`;
- dhe me mesazh, p.sh. `"Ora e futur është:  14:05"`.

Kërkohet: pseudokod + program në C.

Detyra 2: Fatura e thjeshtë

Përdoruesi fut:

- emrin e produktit (pa hapësira),
- çmimin për njësi (`double`),
- sasinë (`int`).

Programi të shfaqë:

- emrin,
- sasinë,
- totalin (`sasi * çmim`) me 2 shifra pas presjes.

Kërkohet: pseudokod, bllokskemë, kod në C.

Detyra 3: Konvertimi i temperaturës

Lexoni C dhe llogaritni $F = \frac{9}{5}C + 32$. Shfaqni të dy vlerat në një mesazh të vetëm. **Kërkohet:** pseudokod, flowchart, C.

Detyra 4: Shuma dhe mesatarja

Lexoni 3 numra të plotë. Shfaqni:

- shumën e tyre;
- mesataren (tip `double`).

Kërkohet: pseudokod, flowchart, program në C.

Detyra 5: Pjesëtimi dhe mbetja

Lexoni `a` dhe `b`.

- Nëse `b != 0`: shfaqni `a/b` dhe `a%b`.
- Nëse `b == 0`: shfaqni mesazh gabimi.

Kërkohet: pseudokod me `if`, flowchart, C.

1 Pse po mësojmë C-në?

2 Programe në C

3 Funksioni printf

4 Funksioni: scanf

5 Operatorët

6 Ushtrime/Detyra

7 Përmbledhje

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.
2. Si kthehet `.c` në program ekzekutues.

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.
2. Si kthehet `.c` në program ekzekutues.
3. Struktura minimale e një programi C.

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.
2. Si kthehet `.c` në program ekzekutues.
3. Struktura minimale e një programi C.
4. Përdorimi i `printf` dhe formateve.

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.
2. Si kthehet `.c` në program ekzekutues.
3. Struktura minimale e një programi C.
4. Përdorimi i `printf` dhe formateve.
5. Sekuencat `escape`.

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.
2. Si kthehet `.c` në program ekzekutues.
3. Struktura minimale e një programi C.
4. Përdorimi i `printf` dhe formateve.
5. Sekuencat `escape`.
6. Përdorimi i `scanf` dhe simbolit `&`.

Çfarë morëm nga ky kapitull

1. Konteksti historik dhe standardet e C-së.
2. Si kthehet `.c` në program ekzekutues.
3. Struktura minimale e një programi C.
4. Përdorimi i `printf` dhe formateve.
5. Sekuencat `escape`.
6. Përdorimi i `scanf` dhe simbolit `&`.
7. Operatorët aritmetikë dhe ++/−.

Si të vazhdoni

- Praktikoni shembujt në kompjuter, jo vetëm në letër.

Si të vazhdoni

- Praktikoni shembujt në kompjuter, jo vetëm në letër.
- Mos e anashkaloni pseudokodin dhe mendimin algoritmik.

Si të vazhdoni

- Praktikoni shembujt në kompjuter, jo vetëm në letër.
- Mos e anashkaloni pseudokodin dhe mendimin algoritmik.
- Në kapitujt pasues: struktura kontrolli, funksione, arrays, pointer-a.

Faleminderit për vëmendjen!

Pyetje?